LevelForge: An Agentic Digital Twin Framework for Feedback-Driven Autonomous Game-Level Generation and Optimization

Swastik Dasgupta, [Team member 2], [Team member 3], [Team member 4]
Department of Artificial Intelligence and Machine Learning
M S Ramaiah Institute of Technology, Bengaluru, India
[email addresses]

***Abstract*—Players often complain that a particular game level is too hard, confusing or unfair, but turning those complaints into a better level is slow, manual work that needs many rounds of play-testing. We present LevelForge, a framework that takes a problem level and its player reviews, places the level in a sandboxed *digital twin*, and lets AI agents redesign it. A large language model (LLM) first mines the reviews into a taxonomy of design problems and derives measurable design targets. *Design agents* (generator, critic and optimiser, orchestrated as a LangGraph state machine) then propose alternative structured edits, and every candidate level is play-tested by hundreds of *player agents* with three distinct play styles. Because each simulated playthrough is independent, play-testing runs simultaneously on a multicore process pool, evaluating up to 11,250 playthroughs per optimisation run in seconds. The developer receives the optimised level, the reasons behind each accepted change, and per-player-type statistics. On 12 starting levels with an identical simulation budget, critic-guided optimisation raised mean quality from 0.572 to 0.783 (+37%) and beat random search on 9 of 12 levels (Wilcoxon $p = 0.034$), matching random search's final quality with two thirds of the budget. Parallel play-testing reached a $3.71\times$ speedup on 5 workers and $6.47\times$ on 12 logical cores, with an experimentally determined serial fraction of 0.086. LLM-driven design agents performed comparably to hand-written rules while also designing levels from scratch and explaining their changes in natural language.**

***Index Terms*—procedural content generation, agentic AI, large language models, digital twin, automated play-testing, multicore parallelism**

I. Introduction

Game developers receive a constant stream of player feedback. On Steam alone, reviews are numerous, and negative reviews in particular contain specific, actionable complaints [1]: "the second floor is way too hard", "I kept getting lost", "traps on the only path are unfair". Fixing a level in response is still largely manual. A designer has to interpret the complaints, guess which change would address them, edit the level, and play-test it again, often many times, because a change that fixes one complaint can cause another.

This paper asks whether that loop can be automated. Our approach follows the workflow a designer would use, but replaces each manual step with an AI component:

1) **Listen:** an LLM reads the player reviews of a problem level and turns them into measurable design targets (*Generative AI*).
2) **Sandbox:** the level is loaded into a digital twin [2] of the game, a fast, simplified simulation that keeps the level layout and game values.
3) **Redesign:** design agents diagnose the level's weaknesses and propose several alternative edits per round (*agentic AI*).
4) **Play-test:** hundreds of player agents play every candidate level simultaneously on all CPU cores (*multicore architecture*).
5) **Report:** the developer receives the optimised level together with an explanation of what changed and why, so the fix can be reviewed and learned from rather than applied blindly.

LLMs can already generate game levels [3], [4], but generation alone does not guarantee that a level is playable, balanced or aligned with what players asked for. Judging that needs many playthroughs, which is computationally expensive. LevelForge therefore combines LLM reasoning with large-scale parallel simulation, and keeps the LLM in a reasoning role while a deterministic validator guarantees that every level stays playable.

Contributions. (i) A review-to-brief pipeline that converts free-text player feedback into quantitative design targets and objective weights. (ii) A two-tier multi-agent system: LLM *design agents* in a generate–simulate–critique–optimise loop with structured, validated edits, simulation feedback and a reflection memory; and rule-based *player agents* that play-test in the digital twin. (iii) A parallel Monte-Carlo play-testing engine on a multicore process pool, with a scaling and task-granularity study. (iv) A controlled comparison of rule-based reasoning, LLM reasoning and random search under the same simulation budget and starting levels. (v) A developer-facing output: a report, natural-language designer notes, and a playable first-person 3D game built from the generated levels.

II. Related Work

Procedural content generation (PCG). Search-based PCG optimises content against an evaluation function, often with evolutionary search [5], [6]. Experience-driven PCG models the player's experience and generates content to shape it [7]. We follow the search-based framing, but derive the evaluation

TABLE I
THE TWO KINDS OF AGENTS IN LEVELFORGE

Agent	Implemented by	Role
Design agents (decide what to change)		
Generator	LLM / rules	load the problem level, or design a first level from the brief
Critic	LLM / rules	diagnose the level's problems from simulation metrics; avoid priorities whose fixes failed
Optimizer	LLM / rules	propose k alternative plans of structured edits
Finalizer	LLM	explain the accepted changes to the developer
Player agents (measure whether it worked)		
Rusher	cost-based planner	shortest path, ignores danger
Cautious	cost-based planner	avoids hazards, heals when low
Explorer	cost-based planner	collects items, then exits

function from player reviews, and use an agent loop with diagnosis instead of blind variation.

Machine learning and LLMs for levels. PCG via machine learning learns content from existing examples [8]. Fine-tuned language models generate Sokoban levels [3] and Super Mario levels from text prompts [4]. These works focus on generating new levels; we focus on *repairing* an existing problem level in response to feedback, with simulation in the loop.

Automated play-testing. Procedural personas use agents with different objectives to represent different play styles [9]. Our player agents follow this idea with three cost-based planning personas, and use the gap between them as a measure of skill expression.

LLM agents and annotation. ReAct interleaves reasoning and actions [10], and Reflexion keeps a verbal memory of failed attempts [11]; our critic's memory of failed fixes is a lightweight form of the latter. LLMs have been shown to match or outperform crowd workers on text annotation [12], which motivates LLM-based review labelling.

Parallel performance. We analyse scaling with Amdahl's law [13] and the Karp–Flatt experimentally determined serial fraction [14].

III. SYSTEM OVERVIEW

Fig. 1 shows the pipeline. Its input is a level and the player reviews of that level; alternatively, the generator agent can design a first level from the reviews alone. Two kinds of agents work together (Table I): design agents decide *what to change*, and player agents measure *whether the change worked*.

IV. METHOD

A. Review Mining and the Design Brief (GenAI)

Reviews are sent to the LLM in batches, which labels each review with zero or more problems from a fixed taxonomy: *too hard*, *too easy*, *repetitive*, *confusing layout*, *unfair traps* and *little reward for exploring*, plus praised aspects. Batches are sent concurrently with a thread pool, because the calls are I/O-bound; a keyword classifier is the offline fallback. Let s_x be the share of reviews that mention problem x, and $d = s_{\text{hard}} - s_{\text{easy}}$. The brief sets a target completion rate and a target share of health lost by players who finish:

$$c^* = \text{clip}(0.65 + 0.35\,d,\ 0.45,\ 0.85), \tag{1}$$

$$h^* = \text{clip}(0.40 - 0.30\,d,\ 0.20,\ 0.60), \tag{2}$$

and raises the weight of the quality component each complaint relates to: engagement by $0.4\,(s_{\text{repetitive}} + s_{\text{empty}})$, flow by $0.4\,s_{\text{confusing}}$ and skill by $0.4\,s_{\text{unfair}}$. Table II shows the result for the 48 sample reviews used in this paper: *too hard* was the most common complaint (23%), so the brief asks for a level that more players finish ($c^* = 0.71$ instead of the default 0.65) and that costs them less health ($h^* = 0.35$ instead of 0.40). Each complaint is also mapped to a likely cause and a candidate design change (e.g. *unfair traps* $\rightarrow$ traps on forced routes $\rightarrow$ move traps to optional routes); these are included in the generator's LLM prompt and in the report.

TABLE II
REVIEW MINING: COMPLAINTS IN THE 48 SAMPLE REVIEWS AND THE RESULTING DESIGN BRIEF

Problem (taxonomy)	Share	Affects
Too hard / punishing	22.9%	$c^* \uparrow$, $h^* \downarrow$
Confusing layout / backtracking	16.7%	flow weight
Repetitive / lacks variety	14.6%	engagement weight
Unfair traps on forced routes	10.4%	skill weight
Little reward for exploring	8.3%	engagement weight
Too easy / no challenge	6.2%	$c^* \downarrow$, $h^* \uparrow$

Brief: $c^* = 0.708$, $h^* = 0.35$; weights balance 0.35, engagement 0.29, flow 0.22, skill 0.19, tension 0.15 (defaults 0.35/0.20/0.15/0.15/0.15)

B. Digital Twin

A level is a character grid of walls, floor, start, exit, key, locked door, monsters, traps, health potions and coins. A turn-based simulator implements the rules: monsters have 3 health and hit with probability 0.6; a monster may strike a player walking past it (probability 0.25); traps trigger with probability 0.6 for 3 damage; potions heal 4; players start with 10 health (maximum 12). A validator checks that a level is well-formed and that the exit is reachable (a door needs a key reachable before it).

The twin mirrors a real-time game, *LevelForge: Breach*: a first-person 3D shooter built with Three.js that runs in the browser. Its levels are built from the same grids and use the same values (monsters become hunter drones, traps become proximity mines, potions become med-kits, coins become data cores). Real-time aiming, dodging and enemy movement are abstracted into discrete encounters so that thousands of playthroughs take seconds. All quality numbers in this paper are measured on the twin. Fig. 2 shows the game.

C. Player Agents

Each player agent plans with Dijkstra search over a persona-specific cost model, re-planning as the level changes (picked-

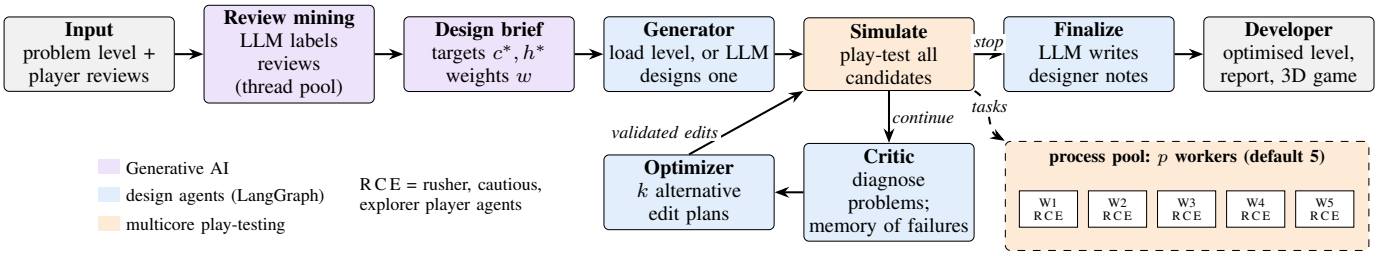


Fig. 1. LevelForge pipeline. Player reviews become a design brief (GenAI). Design agents run a LangGraph loop: the critic diagnoses the current best level, the optimiser proposes k alternative edit plans, and all candidates are play-tested by player agents spread over a multicore process pool. The loop stops after a fixed number of rounds and the developer receives the result.

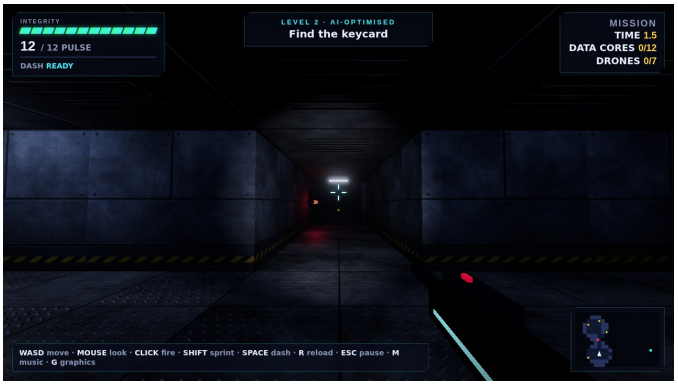


Fig. 2. *LevelForge: Breach*, the playable first-person game built from the twin's levels (level 2, the AI-optimised level). The twin's monsters appear as hunter drones (red, far end of the corridor); the minimap at the bottom right shows the same grid as Fig. 5.

up items, defeated monsters). The *rusher* pays almost nothing for danger, the *cautious* agent pays heavily for traps and monsters and detours to heal, and the *explorer* collects coins before exiting. Randomness in fights and traps comes from a seeded generator, so each playthrough is defined by the tuple (level, persona, seed).

D. Metrics and Quality Score

For each level we aggregate completion, death and timeout rates, damage, backtracking ratio β (steps that revisit tiles), exploration x, coin collection κ by explorers, health lost by finishers h, and completion per persona c_p. Five components in $[0, 1]$ are

$$q_{\text{bal}} = 1 - \frac{|c-c^*|}{\max(c^*, 1-c^*)}, \qquad q_{\text{eng}} = 0.6\,\kappa + 0.4\min(1, x/0.6),$$
$$q_{\text{flow}} = 1 - 2\beta, \qquad q_{\text{skill}} = \frac{c_{\text{cautious}} - c_{\text{rusher}}}{0.35},$$
$$q_{\text{ten}} = 1 - \frac{|h-h^*|}{\max(h^*, 1-h^*)},$$

each clipped to $[0, 1]$, and the quality score is the weighted mean $Q = \sum_k w_k q_k / \sum_k w_k$ with brief-derived weights. Balance penalises both under- and overshooting the target; skill expression rewards levels where careful play beats reckless play.

Algorithm 1 Agent loop for one problem level

Require: level L_0 (or brief only), brief B, rounds R, plans k, runs n

$L^\star \leftarrow$ GENERATOR(L_0, B); $Q^\star \leftarrow$ SIMULATE($\{L^\star\}$)
memory $\leftarrow \emptyset$
for $r = 1$ to R **do**
  $D \leftarrow$ CRITIC(metrics($L^\star$), B, memory)
  $\{P_1..P_k\} \leftarrow$ OPTIMIZER(D, last round's scores)
  $C_i \leftarrow$ APPLYVALIDATED($L^\star$, P_i) for $i = 1..k$
  scores $\leftarrow$ SIMULATE($\{C_1..C_k\}$) ▷ $k \cdot 3n$ runs, parallel
  if $\max_i Q(C_i) > Q^\star$ **then**
    $L^\star, Q^\star \leftarrow$ best candidate ▷ accept
  else
    memory $\leftarrow$ memory $\cup\{D.\text{priority}\}$ ▷ reject, roll back
  end if
end for
return $L^\star$, FINALIZE(history)

E. Design Agents and the Agent Loop

The loop is a LangGraph [15] state graph (Algorithm 1). The **generator** either loads the problem level supplied by the developer or asks the LLM to write a grid from the brief; an invalid LLM grid gets one self-correction attempt, then deterministic repair. The **critic** diagnoses the best level so far using a fixed vocabulary of problem identifiers (e.g. *too_hard*, *low_skill_expression*) and the metrics behind them, and keeps a memory of priorities whose fixes were rejected, moving to a different priority next time. The **optimizer** proposes $k = 4$ alternative plans of structured edits: add or remove a monster, trap, potion or coin on or off the main route; open a loop; add or remove a lock. It also receives the simulated scores of the previous round's plans and which plans could not be applied. The **finalizer** writes designer notes.

The LLM never edits the grid directly. Edits are validated and applied deterministically, so the agents' reasoning is LLM-driven but every resulting level is guaranteed playable. The rule-based variant replaces the LLM critic and optimizer with a hand-written diagnosis and playbook.

F. Parallel Play-Testing (Multicore)

Every playthrough (level, persona, seed) is an independent task, so play-testing is embarrassingly parallel: workers never communicate, and the only synchronisation is the final gather. In each round the tasks of all k candidates are combined into

one sweep of $k \cdot 3n$ tasks over a persistent process pool (default $p = 5$). Processes are used instead of threads because the simulator is CPU-bound Python and the global interpreter lock would serialise threads (review labelling, which waits on the network, uses threads). The pool is created once per run, so its start-up cost is paid once. All candidates use the same seeds (common random numbers [16]), so score differences come from the level, not from luck. Tasks are dispatched in small chunks (at most 8), a choice based on the granularity study in Section VI-D.

G. What the Developer Receives

The goal is a level the developer can *learn from*, not just a new grid. Each run produces an HTML report with the review analysis, the score per round, every candidate the agents tried with its score, the accepted plan per round, before/after levels, animated replays of each player agent, and designer notes (Table V is an excerpt). Finally, the levels are exported into the playable 3D game, so the team can experience the problem level and its optimised version directly. For the case study of Section VI-C, the notes read:

> *"The initial level scored 0.48; after optimisation it scores 0.89. Completion moved from 32% to 72% against a review-derived target of 71%. Backtracking went from 12% to 4% of steps, and explorers now collect 100% of the coins (was 77%). Per-player-type completion (rusher / cautious / explorer): 31%/31%/34% → 66%/100%/51%."*

These notes come from the rule-based finalizer, which fills a template; the LLM finalizer writes free-form notes that also explain *why* each change helped.

H. Implementation

LevelForge is about 2,800 lines of Python. The agent loop uses LangGraph; parallel play-testing uses the standard `multiprocessing` pool; the LLM client uses only the standard library and talks to an OpenAI-compatible endpoint (Groq) in JSON mode, with retries that honour the server's `Retry-After` header. If the API key is missing or a call fails permanently, every LLM step falls back to its rule-based counterpart, so a demonstration cannot break mid-run. All randomness is seeded, so runs are reproducible: for example, `run_demo.py --offline --level data/problem_level.txt` reproduces the case study exactly. The game is a single offline HTML file (Three.js bundled with esbuild) generated from the runs' saved data, so it needs no server and no re-simulation.

V. Experimental Setup

Hardware. A laptop with [CPU model], [N] physical cores and 12 logical cores, [RAM] GB RAM, Windows, Python [version]. The GPU was not used for the experiments.

LLM. GPT-OSS-120B [17] served through the Groq API, with JSON output mode and low reasoning effort. Every LLM step falls back to rules if a call fails.

Optimisation study. Since real reviewed levels were not available, 12 procedurally generated levels serve as stand-ins for problem levels (their mean completion of 60% is well below the 71% target). All methods start from the same level and get the same budget: 6 rounds × 4 candidates × 300 playthroughs. Methods: no optimisation (raw), random search (same loop with random edits and no critic), and the agent loop with rule-based reasoning.

LLM study. The first 4 of those levels, adding the agent loop with LLM reasoning (99 API calls, 425 s in total). It is smaller because of free-tier API rate limits.

Statistics. Paired comparisons per level: exact two-sided Wilcoxon signed-rank test [18] and sign test.

Scaling study. 3,600 playthroughs (6 levels × 3 personas × 200) at 1–12 workers, best of 3 repeats, chunk size $n/(4p)$; then chunk sizes 1, 8, 180 and 720 at 5 workers.

TABLE III
Optimisation quality on 12 levels (same budget; completion target 71%)

Method	Q (mean ± sd)	Completion
No optimisation (raw)	0.572 ± 0.093	60%
Random search	0.758 ± 0.064	78%
Agent loop, rule-based	$\mathbf{0.783 \pm 0.061}$	74%

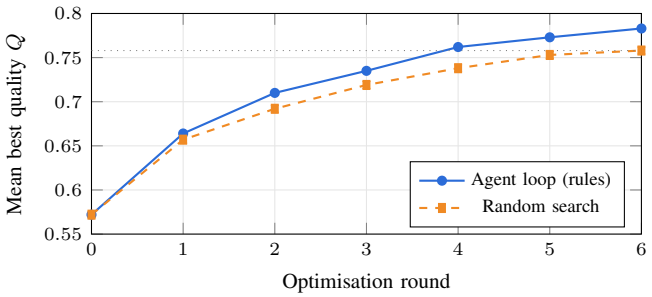


Fig. 3. Convergence over 12 levels. The dotted line is random search's final quality, which the agent loop reaches by round 4.

VI. Results

A. Optimisation Quality

Table III summarises the 12-level study. Both optimisers improved every level over the raw level (Wilcoxon $p < 0.001$ each): +32% for random search and +37% for the agent loop. The agent loop beat random search on 9 of 12 levels, with a mean paired gain of +0.026 (Wilcoxon $p = 0.034$). The sign test is weaker ($p = 0.146$) because it ignores the size of each difference: the agent's wins (up to +0.089) were larger than its losses (at most −0.020). The agent loop also landed closer to the completion target (74% vs. 78%).

Fig. 3 shows the best quality per round. The agent loop reached random search's final quality (0.758) by round 4, i.e. with about two thirds of the simulation budget.

B. LLM vs. Rule-Based Design Agents

On the 4-level subset (Table IV) the LLM agent won 2 of 4 levels against random search and 2 of 4 against the rule-based agent. With 4 levels no difference is statistically detectable (the smallest possible two-sided sign test p-value is 0.125), so

TABLE IV
LLM VS. RULE-BASED REASONING (4 LEVELS)

Method	Quality Q (mean $\pm$ sd)
No optimisation (raw)	0.631 ± 0.012
Random search	0.757 ± 0.053
Agent loop, rule-based	$\mathbf{0.787 \pm 0.026}$
Agent loop, LLM (GPT-OSS-120B)	0.766 ± 0.082

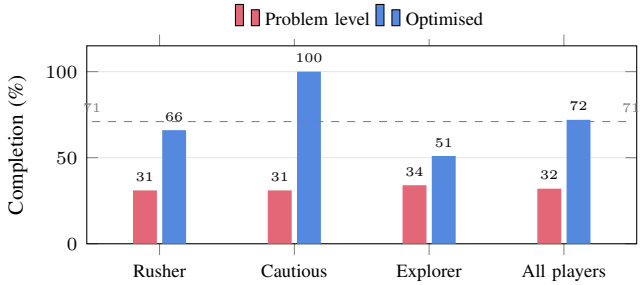


Fig. 4. Completion per player agent in the case study. Before optimisation all play styles did equally badly, so skill did not matter; afterwards careful play is rewarded and overall completion is close to the review-derived target (dashed, 71%).

we report the methods as comparable. The LLM agent had the highest variance: the best result on two levels (0.804, 0.844) and the worst agent result on another (0.655).

C. Case Study: Repairing a Problem Level

Fig. 5 shows a problem level before and after optimisation, and Table V shows what the developer sees round by round. Only 32% of playthroughs reached the exit (target 71%) and, worse for design, reckless and careful players did equally badly (31% vs. 31%), so skill was not rewarded. After six rounds and 11,250 simulated playthroughs (19 s on 2 workers), quality rose from 0.477 to 0.895: completion 72%, backtracking down from 12% to 4% of steps, and per-persona completion (rusher/cautious/explorer) changed from 31/31/34% to 66/100/51%, so careful play now pays off. Only 12 tiles changed, which makes the fix easy for a designer to review. The component scores moved as follows: balance 0.45→0.98, engagement 0.62→0.86, flow 0.77→0.92, skill 0.02→0.97 and tension 0.42→0.63; tension improved least, because finishers still lose 59% of their health against a target of 35%. Fig. 4 shows the per-persona effect, which is the part of the result a designer is most likely to act on.

Two further runs show the LLM in its generative role. In the full LLM pipeline, the LLM designed a corridor-based level from the brief alone (quality 0.604, 100% completion, i.e. too easy); the LLM agents then raised it to 0.879 with 79% completion, mainly by placing hazards in corridors where players cannot walk around them. Starting from the case-study level, however, the LLM agents reached only 0.750 (vs. 0.895 for rules): they repeatedly added potions to reduce health loss (78% vs. the 35% target), but each potion pushed completion above its target, so four consecutive rounds were rejected. This illustrates a limitation of LLM planning with

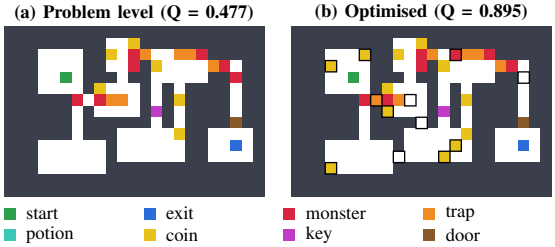


Fig. 5. Case study level (a) before and (b) after optimisation by the rule-based agents. Outlined tiles in (b) were changed. The run can be reproduced with `run_demo.py --level data/problem_level.txt`.

TABLE V
ACCEPTED CHANGE PER ROUND (CASE STUDY)

Rd	Accepted plan (problem it targets)	Q	Compl.
0	problem level	0.477	32%
1	+1 monster on route, open a loop (skill)	0.717	72%
2	+1 trap on route, open a loop (skill)	0.769	62%
3	+2 coins off route (engagement)	0.848	61%
4	-1 trap, -1 monster on route (too hard)	0.876	76%
5	+2 coins off route (exploratory plan)	0.890	72%
6	+2 coins off route (exploratory plan)	0.895	72%

competing objectives, even when the scoring rules are given in the prompt.

D. Parallel Scaling

Table VI and Fig. 6 report speedup $S_p = T_1/T_p$, efficiency S_p/p and the Karp–Flatt metric $e = (1/S_p - 1/p)/(1 - 1/p)$ [14]. Throughput rose 6.5× from 1 to 12 workers; pool start-up cost 0.13–0.22 s and is paid once per run. The serial fraction e stayed roughly constant (0.074–0.113, mean 0.086). A constant e points to a fixed serial portion (task creation, pickling levels, aggregating results) rather than overhead that grows with the number of workers. Amdahl's law [13] with $f = 0.086$ bounds the speedup at about 11.6×. Efficiency falls faster beyond [N physical cores] workers because extra workers share physical cores through hyper-threading [and the CPU mixes performance and efficiency cores, if applicable].

Task granularity. At 5 workers, throughput was 1,205 sims/s with chunk size 1, 1,168 with 8, 997 with 180 ($= n/4p$) and 909 with 720 ($= n/p$). Contrary to the common advice to batch small tasks, the smallest chunks were fastest (+21% over $n/4p$). A playthrough costs about 3.7 ms, much more than dispatching it, but its cost varies between levels and personas, so large chunks leave some workers idle at the end of a sweep (load imbalance). We therefore changed the default to small chunks (at most 8). The scaling table above used the earlier $n/4p$ setting, so it is a conservative estimate.

VII. DISCUSSION AND LIMITATIONS

The agent loop's main benefit over random search is efficiency: equal quality with about a third fewer simulations, and a modest but consistent gain at equal budget. Its accepted

TABLE VI
PARALLEL SCALING, 3,600 PLAYTHROUGHS

p	Time (s)	Sims/s	Speedup	Eff.	e
1	13.36	269	1.00	100%	–
2	7.44	484	1.80	90%	0.113
3	5.30	679	2.52	84%	0.095
4	4.22	853	3.16	79%	0.088
5	3.60	999	3.71	74%	0.087
7	2.75	1307	4.85	69%	0.074
9	2.44	1477	5.48	61%	0.080
11	2.12	1701	6.31	57%	0.074
12	2.07	1743	6.47	54%	0.078

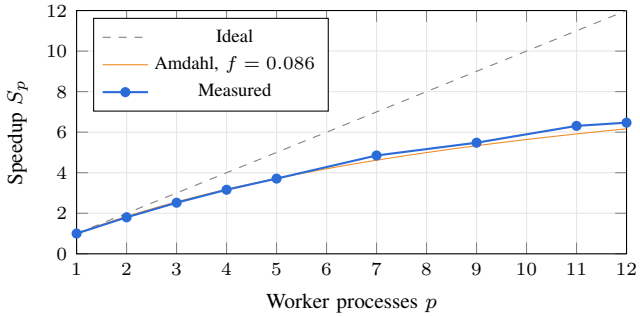


Fig. 6. Measured speedup against ideal linear scaling and Amdahl's law with the measured serial fraction. The demo default is $p = 5$.

changes are also interpretable (Table V), which matters when the goal is a fix that a developer understands. LLM reasoning was comparable to hand-written rules but less consistent; its advantages are designing levels from scratch, natural-language explanations, and needing no hand-written playbook, while its costs are API latency and rate limits (425 s for 4 levels vs. seconds offline).

Why structured edits? LLM-written grids can violate hard constraints (uneven rows, unreachable exits), which is why the generator needs self-correction and deterministic repair. Restricting the LLM to choosing among validated edit operations keeps its reasoning role (what to change, where, and why) while guaranteeing playability, and makes every change small enough for a designer to review.

Generality. The framework needs three things from a game: a level representation that edits can be applied to, a fast simulator with a few player personas, and a complaint taxonomy mapped to metrics. The agent loop, the parallel engine and the statistics are game-independent, so the same approach could apply to, for example, platformer or puzzle levels.

Threats to validity. The player agents are rule-based and only approximate human players; the quality score is a designed proxy and we do not claim it measures fun. The 48 reviews are synthetic and the problem levels in the main study are procedurally generated stand-ins, so claims about real players require real reviewed levels. The twin abstracts real-time shooting into discrete encounters. LLM labelling varies slightly between runs, so each experiment computes the brief once and shares it across methods. The LLM study used only 4 levels.

VIII. CONCLUSION AND FUTURE WORK

LevelForge takes a level that players complain about, turns their reviews into measurable targets, and improves the level through an agentic loop backed by parallel simulated playtesting, returning an optimised level with an explanation the developer can learn from. Critic-guided optimisation beat random search at equal cost, and multicore simulation made thousands of playthroughs per run practical on a laptop. Future work: real reviews paired with real levels, a larger LLM study, learned player models, modelling shooting directly in the twin, multi-objective optimisation (e.g. Pareto search) for the trade-offs the LLM struggled with, and a user study with human players.

REFERENCES

[1] D. Lin, C.-P. Bezemer, Y. Zou, and A. E. Hassan, "An empirical study of game reviews on the Steam platform," *Empirical Software Engineering*, vol. 24, no. 1, pp. 170–207, 2019.

[2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*. Springer, 2017, pp. 85–113.

[3] G. Todd, S. Earle, M. U. Nasir, M. C. Green, and J. Togelius, "Level generation through large language models," in *Proc. 18th Int. Conf. on the Foundations of Digital Games (FDG)*, 2023.

[4] S. Sudhakaran, M. González-Duque, C. Glanois, M. Freiberger, E. Najarro, and S. Risi, "MarioGPT: Open-ended text2level generation through large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-based procedural content generation: A taxonomy and survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.

[6] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016.

[7] G. N. Yannakakis and J. Togelius, "Experience-driven procedural content generation," *IEEE Transactions on Affective Computing*, vol. 2, no. 3, pp. 147–161, 2011.

[8] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen, and J. Togelius, "Procedural content generation via machine learning (PCGML)," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018.

[9] C. Holmgård, M. C. Green, A. Liapis, and J. Togelius, "Automated playtesting with procedural personas through MCTS with evolved heuristics," *IEEE Transactions on Games*, vol. 11, no. 4, pp. 352–362, 2019.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Int. Conf. on Learning Representations (ICLR)*, 2023.

[11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] F. Gilardi, M. Alizadeh, and M. Kubli, "ChatGPT outperforms crowd workers for text-annotation tasks," *Proceedings of the National Academy of Sciences*, vol. 120, no. 30, p. e2305016120, 2023.

[13] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[14] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.

[15] LangChain, "LangGraph: Build resilient language agents as graphs," https://github.com/langchain-ai/langgraph, 2024, accessed Oct. 2026.

[16] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015.

[17] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv:2508.10925, 2025.

[18] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.